



VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

Approved by AICTE, Permanently Affiliated to JNTU, Kakinada

Accredited by NAAC with 'A' Grade - ISO 9001:2008 Certified

Nambur (V), Peda Kakani (M), Guntur Dt. - 522508 April, 2025.

A Project Report

on

Home Rental & PG Finder

submitted for partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

by

NEELURI CHAITHANYA

- 24BQ5A0516

MULUGU SOMA SEKHARA CHARI

- 23BQ1A05E5

POTLA VENKATA VINAY

- 24BQ5A05I1

NUNE REVANTH REDDY

- 23BQ1A05G2

Under the Guidance of

Dr. N. Lakshmi Prasanna

Professor, CSE



VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
(Autonomous)

Permanently Affiliated to JNTU, Kakinada, Approved by AICTE

Accredited by NAAC with 'A' Grade, ISO 9001:2015 Certified

Nambur, Pedakakani (M), Guntur (Dt) - 522508

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

B.Tech Programme Accredited by NBA

CERTIFICATE

This is to certify that the Project Report titled **“Home Rental & PG Finder”** is being submitted by, **Mr. NEELURI CHAITHANYA, Mr. MULUGU SOMA SEKHARA CHARI, Mr. POTLA VENKATA VINAY, Mr. NUNE REVANTH REDDY**, bearing Registered Numbers **24BQ5A0516, 23BQ1A05E5, 24BQ5A05I1, and 23BQ1A05G2** respectively of III B.Tech II semester Computer Science & Engineering is a record Bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Project Guide

(Dr. N. Lakshmi Prasanna, Professor)

Head of Department

(Dr. V. Ramachandran, Professor)

Submitted for Viva-voce Examination held on _____

Internal Examiner

External Examiner

DECLARATION

We, **Mr. NEELURI CHAITHANYA, Mr. MULUGU SOMA SEKHARA CHARI, Mr. POTLA VENKATA VINAY, Mr. NUNE REVANTH REDDY**, hereby declare that the Project Report entitled “HOME RENTAL & PG FINDER” done by us under the guidance of **Dr. N. Lakshmi Prasanna, Professor at Vasireddy Venkatadri Institute of Technology (A)** is submitted in partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science & Engineering. The results embodied in this report have not been submitted to any other university for the award of any degree.

DATE:

PLACE:

SIGNATURE OF THE CANDIDATE(s)

1.

2.

3.

4.

ACKNOWLEDGMENT

We take this opportunity to express my deepest gratitude and appreciation to all those people who made this project work easier with words of encouragement, motivation, discipline, and faith by offering different places to look to expand our ideas and helped us towards the successful completion of this project work.

First and foremost, we express our deep gratitude to **Sri. Vasireddy Vidya Sagar**, Chairman, Vasireddy Venkatadri Institute of Technology (A) for providing necessary facilities throughout the B Tech program.

We express our sincere thanks to **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology (A) for his constant support and cooperation throughout the B. Tech program.

We express our sincere gratitude to **Dr. V. Ramachandran**, Professor & HOD, Computer Science & Engineering, Vasireddy Venkatadri Institute of Technology (A) for his constant encouragement, motivation and faith in offering different places to look to expand our ideas.

We would like to express our sincere gratefulness to our Guide **Dr. N. Lakshmi Prasanna**, Professor, CSE for his insightful advice, motivating suggestions, invaluable guidance, help and support in the successful completion of this project.

We would like to express our sincere heartfelt thanks to our Project Coordinator **Dr. Vidya Sagar Uppuluri**, Professor, CSE for his valuable advices, motivating suggestions, moral support, help and coordination among us in successful completion of this project.

We would like to take this opportunity to express our thanks to all the **Teaching and Non-Teaching** Staff in the Department of Computer Science & Engineering, VVIT for their invaluable help and support.

Names of Students

NEELURI CHAITHANYA	- 24BQ5A0516
MULUGU SOMA SEKHARA CHARI	- 23BQ1A05E5
POTLA VENKATA VINAY	- 24BQ5A05I1
NUNE REVANTH REDDY	- 23BQ1A05G2



VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
(Autonomous)

Permanently Affiliated to JNTU, Kakinada, Approved by AICTE

Accredited by NAAC with 'A' Grade, ISO 9001:2015 Certified

Nambur, Pedakakani (M), Guntur (Dt) - 522508

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

B.Tech Programme Accredited by NBA

INSTITUTE VISSION

To impart quality education through exploration and experimentation and generate socially conscious engineers, embedding ethics and values, for the advancement in Science and Technology.

INSTITUTE MISSION

- To educate students with practical approach to dovetail them to industry needs.
- To govern the institution with a proactive and professional management with passionate teaching faculty.
- To provide holistic and integrated education and achieve over all development of students imparting scientific and technical, social and cognitive, managerial and organizational skills.
- To compete with the best and be the most preferred institution of the studios and the scholarly.
- To forge strong relationships and linkage with the industry.



VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(Autonomous)

Permanently Affiliated to JNTU, Kakinada, Approved by AICTE

Accredited by NAAC with 'A' Grade, ISO 9001:2015 Certified

Nambur, Pedakakani (M), Guntur (Dt) - 522508

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

B.Tech Programme Accredited by NBA

DEPARTMENT VISSION

Providing quality education to enable the generation of socially conscious software engineers who can contribute to the advancement in the field of computer science and engineering.

DEPARTMENT MISSION

- To equip the graduates with the knowledge and skills required to enable them to be industry ready.
- To train socially responsible, disciplined engineers who work with good leadership skills and can contribute for nation building.
- To make our graduates proficient in cutting edge technologies through student centric teaching-learning process and empower them to contribute significantly to the software industry.
- To shape the department into a centre of academic and research excellence.

COURSE OUTCOMES

- CO 1:** Articulate problem statement. (K2)
- CO 2:** Apply technical knowledge. (K3)
- CO 3:** Acquire contemporary tools & technologies. (K2)
- CO 4:** Communicate and present the entire SDLC. (K3)
- CO 5:** Perform the role of a team member or lead in SDLC. (K3)

Program Educational Objectives (PEOs)

The Programme Educational Objectives of the B.Tech in Computer Science Engineering programme are given below and are numbered from PEO1 to PEO4.

PEO1	To provide the graduates with solid foundation in computer science and engineering along with fundamentals of Mathematics and Sciences with a view to impart in them high quality technical skills like modelling, analyzing, designing, programming and implementation with global competence and helps the graduates for life-long learning.
PEO2	To prepare and motivate graduates with recent technological developments related to core subjects like programming, databases, design of compilers and Network Security aspects and future technologies so as to contribute effectively for Research & Development by participating in professional activities like publishing and seeking copy rights.
PEO3	To train graduates to choose a decent career option either in high degree of employability / Entrepreneur or, in higher education by empowering students with ethical administrative acumen, ability to handle critical situations and training to excel in competitive examinations.
PEO4	To train the graduates to have basic interpersonal skills and sense of social responsibility that paves them a way to become good team members and leaders.

Program Outcomes (POs)

The B.Tech CSE program has documented measurable outcomes that are based on the needs of the program's stakeholders. The program outcomes, which are derived from ABET criteria, were first drafted in the academic year 2009–10 and later revised in 2010–11. The program outcomes that the department presently adopts are as follows:

1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural science and engineering sciences.
3	Design/development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal and environmental considerations.
4	Conduct investigations of complex problems	Use research based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5	Modern tool usage	Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6	The Engineer and Society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7	Environment and Sustainability	Understand the impact of the professional engineering solutions in the societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9	Individual and Team Work	Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.
10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11	Project Management and Finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

1 2	Lifelong Learning	Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.
--------	--------------------------	--

Program Specific Outcomes (PSOs)

PSO-1	Professional Skills:	The ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, multimedia, web design, big data analytics and networking for efficient design of computer-based systems of varying complexity.
PSO-2	Successful Career and Entrepreneurship:	The ability to employ modern computer languages, environments, and platforms in creating innovative career paths to be an entrepreneur and a zest for higher studies/employability in the field of Computer Science & Engineering.

CO – PO ARTICULATION MATRIX

Course Outcomes:

CO	Description
CO1	Articulate problem statement. (K2)
CO2	Apply technical knowledge. (K2)
CO3	Acquire contemporary tools & technologies. (K2)
CO4	Communicate and present the entire SDLC. (C3)
CO5	Perform the role of a team member or lead in SDLC. (K3)

	P O 1	P O 2	P O 3	P O 4	P O 5	P O 6	P O 7	P O 8	P O 9	P O 10	P O 11	P O 12	P S O 1	P S O 2
CO 1	1							3					1	
CO 2	2							1	2				3	
CO 3													1	
CO 4	1								3				1	
CO 5								2	3				1	

INDEX

CH No	TITLE	PAGE NO
CHAPTER 1	INTRODUCTION	1
	1.1 Introduction	
CHAPTER 2	AIM AND SCOPE	2-3
	2.1 Existing System	
	2.2 Proposed System	
CHAPTER 3	CONCEPT AND METHODS	4-7
	3.1 Problem Description	
	3.2 Proposed Solution	
	3.2.1 Database Design	
	3.2.2 Data Collection	
	3.2.3 Data Validation and Processing	
	3.2.4 Data Enrichment	
	3.2.5 System Architecture	
	3.2.6 System Evaluation Metrics	
CHAPTER 4	IMPLEMENTATION	8- 18
	4.1 Tools Used	
	4.2 Code	
	4.3 Results and Screenshots	
CHAPTER 5	CONCLUSION AND FUTURE SCOPE	19- 23
	CONCLUSION	
	FUTURE SCOPE	
	REFERENCES	
	APPENDIX	

CHAPTER – 1

INTRODUCTION

1.1 Overview

In the contemporary, fast-paced urban environment, the search for suitable residential accommodation—ranging from independent homes to Paying Guest (PG) facilities—has become a significant challenge for students and working professionals. Despite the proliferation of digital listing platforms, the process of finding a home is often compared to "searching for a needle in a haystack". This difficulty is compounded by a fragmented market where tenants are frequently subjected to endless unorganized listings, fraudulent advertisements, and stagnant communication with property owners.

The Home Rental & PG Finder project is conceived as an intelligent, secure web application designed to bridge the trust and efficiency gap in the rental market. Built using a modern full-stack approach—specifically the MERN stack (MongoDB, Express, React, and Node.js)—the platform serves as a "seamless connection" between property owners and prospective tenants. By integrating advanced system-driven validation, the project aims to redefine the property search experience as one that is simpler, safer, and fundamentally more human-centered.

1.2 Problem Statement

Traditional property listing websites often operate as mere digital bulletin boards, lacking the necessary oversight to ensure data quality. This leads to several systemic failures:

The Trust Deficit: Many platforms do not verify the authenticity of listings, allowing "fake advertisements" to proliferate and waste tenant time.

Stale Data: Properties that have already been occupied often remain listed, leading to "frustrated tenants" and unnecessary inquiries for owners.

Information Overload: Without intelligent filtering or recommendation systems, users are forced to manually sift through thousands of irrelevant properties.

Communication Bottlenecks: Delayed or non-existent responses from property owners create a sense of uncertainty for tenants who need immediate accommodation.

1.3 Rational and Motivation

The motivation behind this project is to transform the rental search from a "frustrating search" into an experience where the website acts as a "helpful assistant" rather than just a list of links. By utilizing Design Thinking and Artificial Intelligence, the system moves beyond passive listings to active, engagement-driven results.

CHAPTER – 2

AIM AND SCOPE

2.1 Aim of the Project

The primary aim of the Home Rental & PG Finder is to establish a trusted, high-engagement marketplace that eliminates the "trust deficit" currently prevalent in the rental industry. The project intends to achieve this by creating an intelligent web ecosystem where every property listing is validated by system administrators and every recommendation is tailored to the specific behavior and preferences of the user.

2.2 Core Objectives

To fulfill the project's aim, the following technical and functional objectives have been established:

System-Driven Validation: To implement a mandatory document upload and verification process for owners to ensure that only authenticated properties are visible to the public.

Intelligent Discovery: To develop an AI-powered matching algorithm that analyzes user preferences, location data, and past interactions to recommend the most suitable homes automatically.

Dynamic Data Management: To ensure "freshness" by automatically identifying and removing inactive or unresponsive listings after 30 days of inactivity.

Engagement Tracking: To rank owners based on their responsiveness and positive feedback, thereby incentivizing better communication between market participants.

User-Centric Design: To provide a "Mobile-First" responsive interface that allows for easy searching, side-by-side property comparisons, and direct communication.

2.2 PROPOSED SYSTEM

2.1 Functional Scope

The scope of the application covers four critical modules that handle the lifecycle of a rental transaction:

A. Search & Discovery Portal

This module focuses on the tenant's first interaction. It includes a "Smart Search Bar" for filtering by budget, location, and type (Home or PG), alongside "Property Showrooms" that display verified photos. A unique "Comparison Tool" is included to allow tenants to evaluate up to three properties side-by-side.

B. Trust & Identity Management

Acting as the platform's "Gatekeeper," this module facilitates secure owner registration and document submission. It includes an Admin Dashboard where property deeds and IDs are checked before a "Trust Badge" is awarded to the listing.

C. AI Matchmaker (The "Brain")

The AI module continuously learns from user behavior. Its scope includes generating smart recommendations and maintaining the "Freshness Filter," which hides "dead" listings to ensure search results remain reliable.

D. Communication Hub

This module establishes the "Seamless Connection" via an inquiry system that allows tenants to send direct "Interest Requests". It also manages periodic notifications (emails or pop-ups) and engagement tracking for owners.

2.2 Technical Scope

Architecture: The system utilizes a client-server architecture with a React.js frontend for interactivity and a Node.js/Express.js backend for business logic.

Data Management: A MongoDB database is used for flexible storage, while external interfaces like Cloudinary/AWS S3 handle high-resolution image storage.

Security: The scope includes implementing JWT (JSON Web Tokens) for session management and bcrypt for password hashing.

2.3 Future Enhancements (Extended Scope)

The project is designed with modularity to allow for future additions, such as:
Digital Rental Agreements: Automated generation of legal contracts.

In-App Payments: Integrated gateways for security deposits.

Virtual Tours: 360-degree property inspections.

CHAPTER – 3

CONCEPTS AND METHODS

3.1 Problem Description

In today's rental ecosystem, tenants often face significant challenges in finding reliable and verified accommodation. Existing platforms lack trust mechanisms, leading to issues such as fraudulent listings, misleading property information, and poor communication between tenants and property owners. Additionally, there is no centralized system that ensures document verification, making it difficult for users to rely on the authenticity of listings.

Property owners, on the other hand, face challenges in managing listings, responding to inquiries efficiently, and reaching genuine tenants. The absence of structured communication and engagement tracking further reduces the effectiveness of current systems.

Therefore, there is a need for a secure, efficient, and user-friendly platform that bridges the gap between tenants and property owners while ensuring trust, transparency, and ease of use.

3.2 Proposed Solution

The proposed solution is a **Home Rental & PG Finder Web Application** developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system is designed to function as a trusted marketplace that connects tenants and property owners through a secure and interactive platform.

The application introduces key features such as:

- Verified property listings through document validation
- Advanced search and filtering options
- Direct communication between tenants and owners
- Role-based authentication (Tenant, Owner, Admin)
- Real-time updates and engagement tracking

The system follows a **client-server architecture**, where the frontend provides an intuitive user interface and the backend handles business logic, authentication, and database operations. The application ensures scalability, security, and performance while maintaining a seamless user experience.

3.2.1 Database Design

The application uses **MongoDB** as the primary database with **Mongoose** for schema modeling. The database is designed to handle multiple entities efficiently, ensuring scalability and flexibility.

The key collections include:

- **User Collection**
Stores details of tenants, owners, and admins, including authentication credentials and preferences.
- **Property Collection**
Stores property listings with details such as title, description, price, location, amenities, images, and verification status.
- **Inquiry Collection**
Acts as a communication bridge between tenants and property owners, storing messages and inquiry statuses.
- **Verification Document Collection**
Stores documents uploaded by property owners for admin verification, ensuring trust and authenticity.

The database design ensures:

- Data consistency
- Efficient querying
- Secure data handling

3.2.2 Data Collection

Data in the system is collected from user inputs through various modules of the application:

- **User Registration**
Users provide personal details such as name, email, phone number, and role (Tenant/Owner).
- **Property Listings**
Owners input property details including location, rent, amenities, and images.
- **Verification Documents**
Owners upload government-approved documents for property validation.
- **Tenant Preferences**
Tenants provide preferences such as budget range, preferred location, and property type.

All data is collected through validated forms to ensure accuracy and completeness.

3.2.3 Data Validation and Processing

To maintain data integrity and security, the system implements strict validation and processing mechanisms:

- Input validation using frontend and backend checks
- Email format and password strength validation
- File size and format restrictions for image uploads
- Password hashing using bcrypt for secure storage
- JWT-based authentication for session management

These mechanisms ensure that only valid and secure data is stored in the system.

3.2.4 Data Enrichment

The system enhances collected data to improve user experience and system efficiency:.

- **Preference Matching**
Tenant preferences are used to filter and recommend relevant properties.
- **Status Tracking**
Properties are categorized as Available or Booked to avoid stale listings.
- **Verification Tagging**
Verified properties are marked with trust badges to improve reliability.

This enrichment improves search accuracy and decision-making for users.

3.2.5 System Architecture

The application follows a **three-tier architecture**:

1. Frontend (Presentation Layer)

- Built using React.js and Tailwind CSS
- Provides responsive and interactive UI
- Handles user interactions and API calls

2. Backend (Application Layer)

- Built using Node.js and Express.js
- Handles authentication, business logic, and APIs
- Manages communication between frontend and database

3. Database (Data Layer)

- MongoDB for storing structured and unstructured data
- Mongoose for schema modeling and validation

Additional Integrations:

- Firebase for image storage

- Google Maps API for location services
- JWT for secure authentication

This architecture ensures modularity, scalability, and maintainability.

3.2.6 System Evaluation Metrics

The performance of the system is evaluated based on the following metrics:

1. Performance

- Fast loading time (less than 2 seconds for search results)
- Efficient API response handling

2. Scalability

- Ability to handle multiple users simultaneously
- Efficient database indexing and querying

3. Security

- Encrypted password storage
- Secure authentication using JWT
- Protection against injection attacks

4. Usability

- User-friendly interface
- Mobile-responsive design
- Easy navigation for both tenants and owners

5. Reliability

- Accurate property listings
- Verified data through admin validation
- Consistent system uptime

These metrics ensure that the system meets both functional and non-functional requirements effectively.

CHAPTER – 4

IMPLEMENTATION

4.1 TOOLS USED

4.1 Tools Used

The application is built on the MERN stack, which provides a high-performance, full-stack JavaScript environment.

Core Development Stack

React.js (Frontend Library): Used to build a responsive, next-generation user interface that ensures fast page loading and cross-device compatibility, particularly for mobile users.

Node.js (Runtime Environment): Provides the execution environment for server-side code, allowing the use of JavaScript outside the browser.

Express.js (Backend Framework): A minimal and flexible web application framework for Node.js used to manage API endpoints and business logic.

MongoDB (Database): A NoSQL database used for flexible, scalable storage of user profiles, property listings, and inquiry data.

Supporting Technologies & APIs:

MongoDB Atlas: Used as the cloud-based database-as-a-service to host and manage application data with automated backups.

JSON Web Tokens (JWT): Utilized for secure session management by generating unique, encrypted tokens upon successful user login.

bcrypt: A password-hashing library used to securely encrypt user passwords before they are stored in the database.

Cloudinary / AWS S3: Selected for high-resolution image storage, ensuring property photos are managed efficiently without impacting server performance.

Google Maps API / Mapbox: Integrated to provide location services, allowing tenants to view property positions on interactive maps.

System Logic & Security:

AI Recommendation Service: A logic layer (Python or Node-based) that powers the "AI Matchmaker" by analyzing user behavior to provide personalized recommendations.

Watermarking Logic: A system-driven tool that automatically watermarks uploaded verification documents with "FOR VERIFICATION ONLY" to prevent data misuse.

4.2 CODE STRUCTURE

The AI Resume Analyzer and Job Prediction System is developed using a modular architecture that separates different functionalities into independent components. This modular structure improves system maintainability, scalability, and ease of future enhancements.

Following is the Model Structure for the core entities of the Home Rental & PG Finder:

1. User Model

This model handles authentication and role-based access for Tenants, Owners, and Admins.

User ID: Unique identifier for each account.

Email/Phone: Validated contact information used for login.

Password: Stored as a hashed string using the bcrypt algorithm for security.

Role: An enumeration to distinguish between "Tenant," "Owner," and "Admin" permissions.

Profile Details: Includes name, contact info, and search preferences.

JWT Token: Issued upon successful login to manage secure active sessions.

2. Property Model

This model manages the details of rental listings and their lifecycle status.

Owner Reference: A foreign key linking the property to a specific User ID.

Property Type: Categorized as "PG," "Flat," or "Private Room".

Core Details: Mandatory fields including Title, Rent Price, and detailed Location (City/Area).

Amenities: An object or array containing boolean values for Wi-Fi, AC, Food, Laundry, and Parking.

Images: A list of URLs or paths for high-resolution property photos.

Verification Status: A restricted field (default: 'Pending') that can only be modified by an Admin after reviewing legal documents.

Availability Toggle: A status field to mark a property as "Available" or "Booked" to prevent stale leads.

3. Verification Document Model

This model is used by the "Trust & Identity" module to store sensitive owner documentation.

Property reference: Links the document to the specific property listing.

Document Type: Type of proof provided (e.g., Government ID or Property Deed).

File URL: Secure path to the restricted storage bucket (e.g., AWS S3) where the document is stored.

Watermark Flag: Indicates if the system-driven watermark ("FOR VERIFICATION ONLY") has been applied.

4. Inquiry & Communication Model

This model facilitates the "Seamless Connection" between the two primary user classes.

Tenant ID: Reference to the user expressing interest.

Owner ID: Reference to the property recipient.

Property ID: Links the inquiry to a specific listing.

Message Content: The text of the interest request or direct message.

Timestamp: Used to track owner "Response Rates" for ranking in search results.

5. AI Behavioral Model (Schema-less)

Used by the "AI Matchmaker" to learn from user interactions.

Interaction History: Tracks clicked properties, saved favorites, and search keywords.

Recommendation Score: A dynamic value used to prioritize listings in the "Property Showroom" based on a user's past behavior.

1. Database Models (Backend - MongoDB/Mongoose)

These models define the structure for users and property listings as described in your system design.

User Model (models/User.js):

JavaScript

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // Hashed using bcrypt [cite: 32]
  role: { type: String, enum: ['Tenant', 'Owner', 'Admin'], default: 'Tenant' }, [cite: 115]
  phoneNumber: { type: String },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', UserSchema);
```

Property Model (models/Property.js)

JavaScript

```
const mongoose = require('mongoose');

const PropertySchema = new mongoose.Schema({
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, [cite: 118]
  title: { type: String, required: true },
  description: { type: String },
  rentPrice: { type: Number, required: true }, [cite: 119]
  location: {
    city: { type: String, required: true },
    area: { type: String, required: true },
    address: { type: String }
  },
  propertyType: { type: String, enum: ['Flat', 'PG'], required: true }, [cite: 119]
  amenities: [String], // e.g., ['Wi-Fi', 'AC', 'Food'] [cite: 125]
  images: [String], // URLs to Cloudinary/S3 [cite: 120, 159]
  isVerified: { type: Boolean, default: false }, // Controlled by Admin [cite: 129, 150]
  status: { type: String, enum: ['Available', 'Booked'], default: 'Available' }, [cite: 121]
```



```
verificationDoc: { type: String } // URL to identity proof [cite: 127]
});
```

```
module.exports = mongoose.model('Property', PropertySchema);
```

2. Backend Controller Logic (Express.js)

This snippet demonstrates the logic for filtering properties, supporting your "Search & Discovery" requirements.

Property Search Controller (controllers/propertyController.js) JavaScript

```
exports.searchProperties = async (req, res) => {
  try {
    const { city, minPrice, maxPrice, type } = req.query;
    let query = { isVerified: true }; // Only show verified listings to tenants [cite: 20, 43]

    if (city) query["location.city"] = new RegExp(city, 'i'); [cite: 123]
    if (type) query.propertyType = type; [cite: 124]
    if (minPrice || maxPrice) {
      query.rentPrice = {};
      if (minPrice) query.rentPrice.$gte = Number(minPrice);
      if (maxPrice) query.rentPrice.$lte = Number(maxPrice); [cite: 124]
    }

    const properties = await Property.find(query).populate('owner', 'name email');
    res.status(200).json(properties); [cite: 15]
  } catch (err) {
    res.status(500).json({ message: "Server Error", error: err.message });
  }
};
```

3. Frontend Component (React.js)

This represents the UI for the "Property Card" seen in your search results.

PropertyCard Component (components/PropertyCard.js) JavaScript

```
import React from 'react';

const PropertyCard = ({ property }) => {
  return (
    <div className="property-card">
```

```

    <img src={property.images[0]} alt={property.title} className="thumbnail" />
    <div className="card-content">
      {property.isVerified && <span className="badge">Verified</span>} [cite: 59]
      <h3>{property.title}</h3>
      <p className="location">{property.location.area}, {property.location.city}</p> [cite:
15]
      <p className="price">₹ {property.rentPrice}/mo</p> [cite: 15]
      <div className="amenities">
        {property.amenities.slice(0, 3).map(amt => (
          <span key={amt} className="tag">{amt}</span>
        ))}
      </div>
      <button className="view-details-btn">View Details</button> [cite: 16]
    </div>
  </div>
);
};

```

```
export default PropertyCard;
```

4. Middleware: JWT Authentication

This handles secure session management as required by your project specifications.

Auth Middleware (middleware/auth.js)

JavaScript

```

const jwt = require('jsonwebtoken');

module.exports = function (req, res, next) {
  const token = req.header('x-auth-token');
  if (!token) return res.status(401).json({ msg: 'No token, authorization denied' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET); [cite: 116]
    req.user = decoded.user;
    next();
  } catch (err) {
    res.status(401).json({ msg: 'Token is not valid' });
  }
};

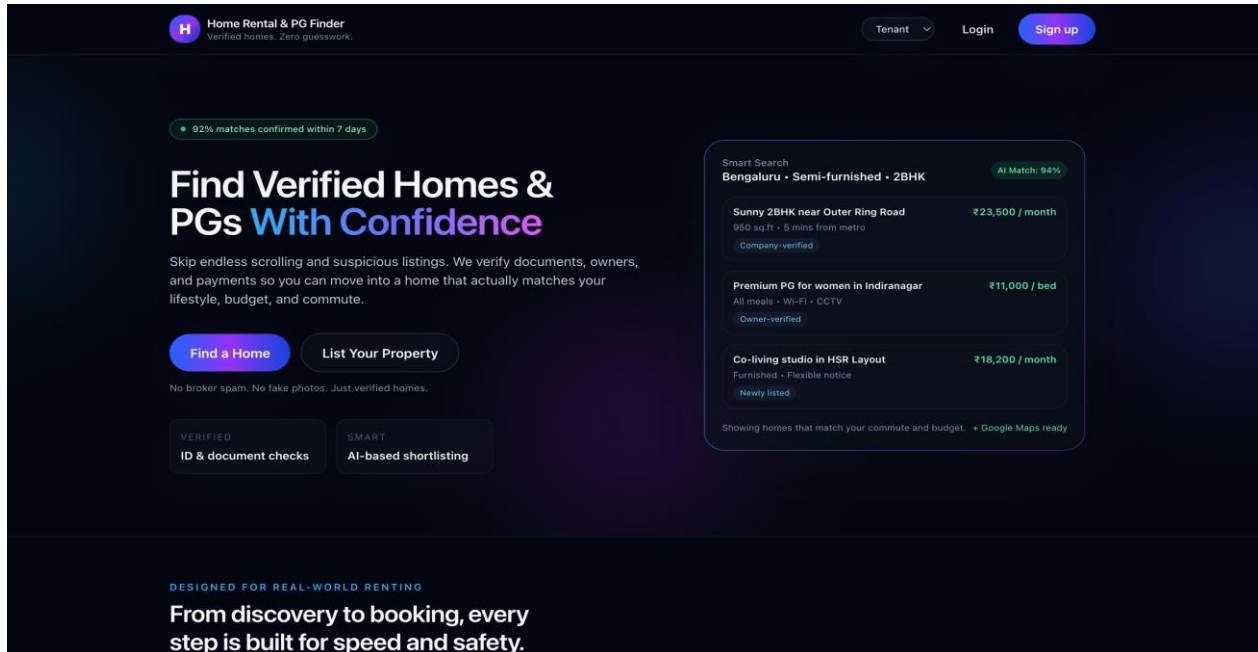
```

4.3 RESULTS AND OUTPUT SCREENSHOTS

After implementing the system, the **Smart Career Readiness Platform** successfully performs the following operations:

1. User Registration and Login

Tenant/Owner can create an account and log in securely to the platform.



Owner/Tenant Registration window:

The screenshot shows the 'Owner Registration' window of the 'Home Rental & PG Finder' application. The header includes the logo, navigation links for 'Tenant', 'Login', and 'Sign up', and a status indicator '92% matches confirmed within 7 days'. The main content area features a large heading 'Owner Registration' and a subheading 'Skip endless scrolling and suspicious listings. We verify documents, owners, and payments so you can move into a home that actually matches your lifestyle, budget, and commute.' Below this are two buttons: 'Find a Home' and 'List Your Property'. A section titled 'DESIGNED FOR REAL-WORLD RENTING' states 'From discovery to booking, every step is built for speed and safety.' On the right, a 'Smart Search' box displays results for 'Bengaluru • Semi-furnished • 2BHK' with an 'AI Match: 94%' score. The results list three properties: 'Sunny 2BHK near Outer Ring Road' (₹23,500 / month), 'Premium PG for women in Indiranagar' (₹11,000 / bed), and 'Co-living studio in HSR Layout' (₹18,200 / month). Each property has a 'Company-verified' or 'Owner-verified' badge and a 'Newly listed' tag. A footer note says 'Showing homes that match your commute and budget. + Google Maps ready'.

Full Name
Your name

Mobile Number
10-digit mobile number

Email
you@yourproperty.com

Password
Create a strong password

Your location
Unable to retrieve your location. You can still continue.

Register as Owner

Already have an account? [Login instead](#)

Verified owners will get priority placement in AI matches once documents are approved by admins.

Looking for a home instead? [Continue as tenant](#)

Owner/Tenant login window

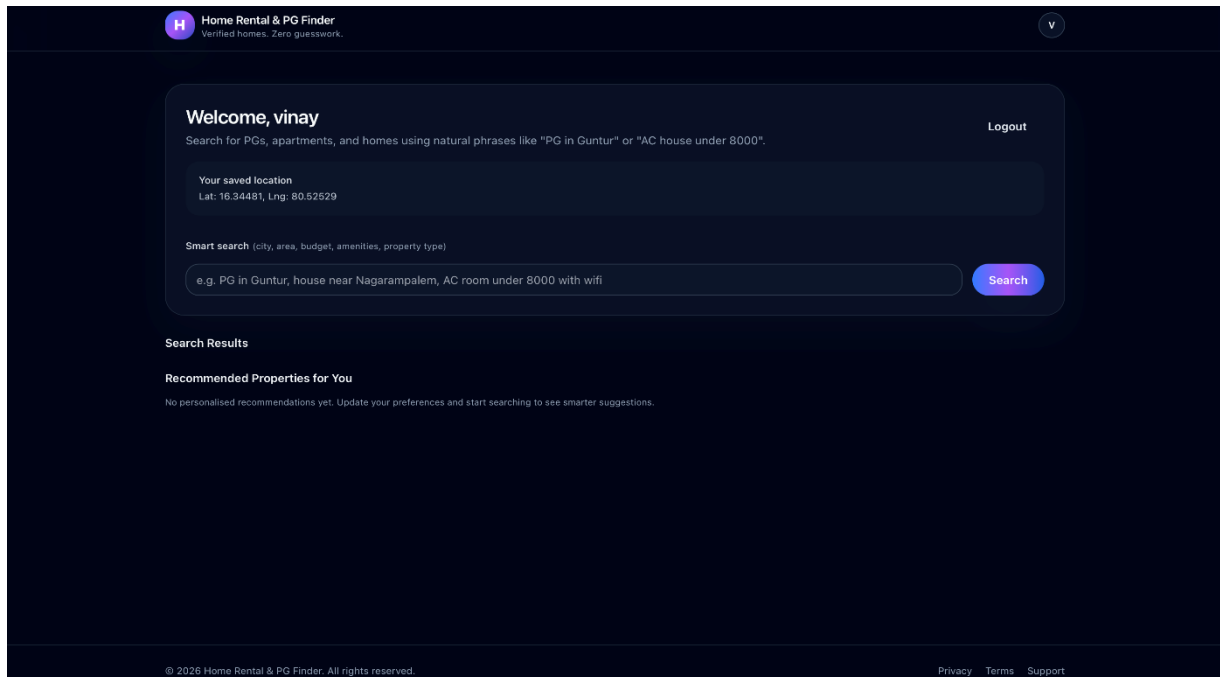
The screenshot shows the 'Tenant Login' window on the Home Rental & PG Finder website. The header includes the logo 'H Home Rental & PG Finder' with the tagline 'Verified homes. Zero guesswork.', a 'Tenant' dropdown menu, a 'Login' link, and a 'Sign up' button. The main content area features a 'Tenant Login' form with fields for 'Email' (containing 'you@example.com') and 'Password' (containing 'Your password'). Below the fields is a large blue 'Login as Tenant' button. At the bottom of the form, there are two links: 'New tenant? Create an account' and 'Looking to list a home instead? Continue as owner'. The footer contains the copyright notice '© 2026 Home Rental & PG Finder. All rights reserved.' and links for 'Privacy', 'Terms', and 'Support'.

2. Profile Creation

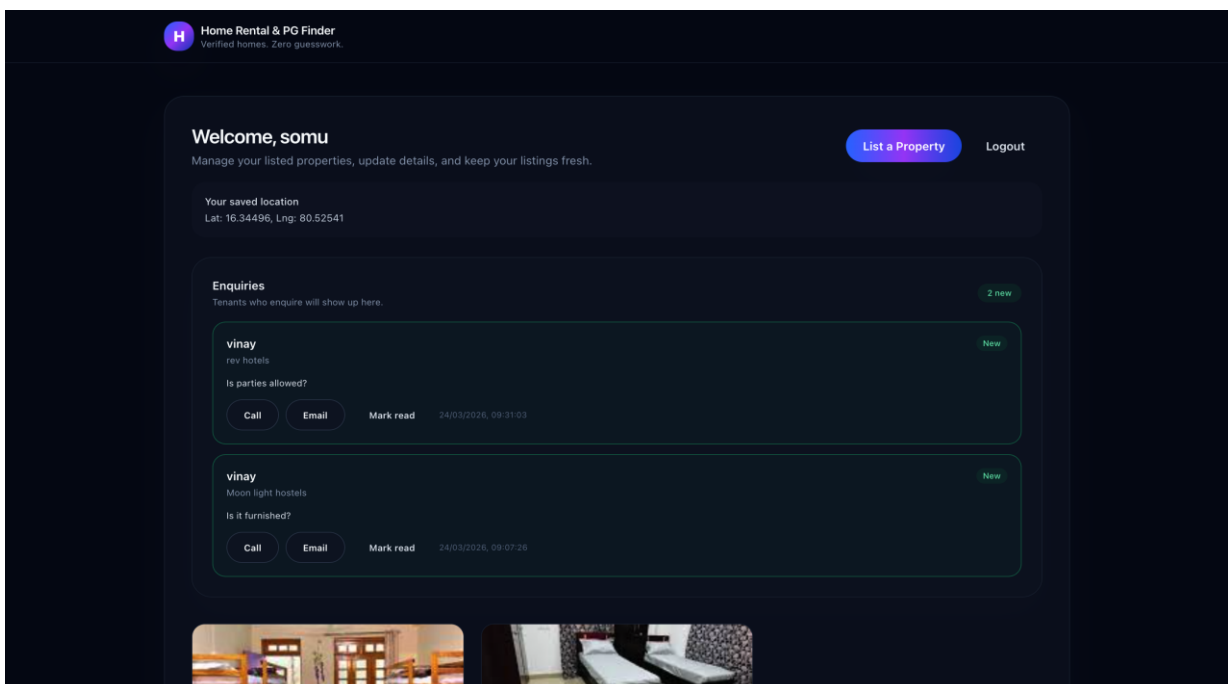
Tenant can enter their details and current interests in their profile.

The screenshot shows the 'Tenant Profile' creation window on the Home Rental & PG Finder website. The header includes the logo 'H Home Rental & PG Finder' with the tagline 'Verified homes. Zero guesswork.', a 'Logout' button, and a 'Tenant' dropdown menu. The main content area features a 'Tenant Profile' form with the subtitle 'Personalize your rental preferences so we can match better homes for you.' The form includes fields for 'Name' (containing 'vinay'), 'Phone Number' (containing '9412345678'), 'Email' (containing 'vinay@gmail.com'), and 'Aadhaar ID'. Below these are sections for 'Accommodation Type' (set to 'PG'), 'Move-in Date' (set to '24/03/2026'), 'Budget Min (₹)' (set to '5000'), and 'Budget Max (₹)' (set to '100000'). There are also fields for 'Preferred City' (set to 'Guntur') and 'Preferred Area' (set to 'Broadipet'). A 'Gender Preference (PG)' dropdown is set to 'Male'. At the bottom, there are 'Amenities' checkboxes for 'WiFi', 'Food', 'Parking', 'Housekeeping', 'Laundry', 'AC', and 'Non-AC'. A large blue 'Save Changes' button is at the bottom of the form.

Dashboard of Tenant:



3. Dashboard of Owner:



Dashboard of Owner while listing his/her property through valid details:

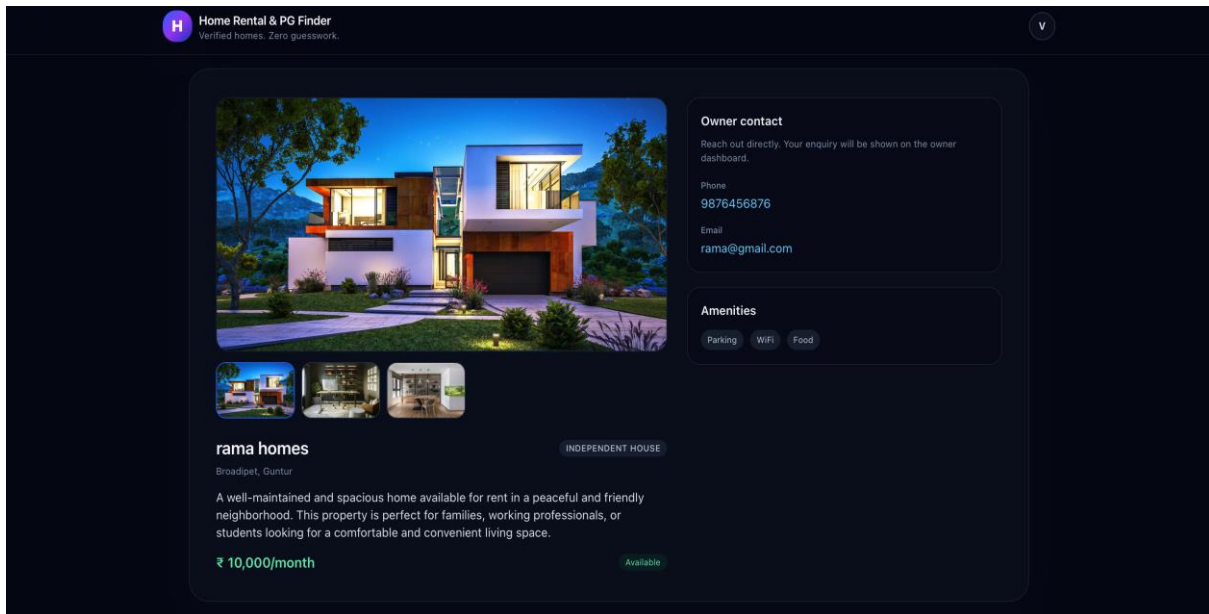
The screenshot shows a web form titled "List a PG / Home / Property" with a sub-header "Create a clean listing with photos, amenities, and documents. Tenants will only see verified, complete listings." The form is divided into several sections: "Basic Information" with fields for "Property Title" and "Description"; "Property Type" with a dropdown menu and "Price per month (₹)" with a text input; "Location" with fields for "City", "Area", "Address", "Pincode", "Latitude (optional)", and "Longitude (optional)"; and "Amenities" with a row of buttons for "WiFi", "Parking", "AC", "Food", "Laundry", "CCTV", "Power Backup", and "Water Supply".

Owner can also update the existing property details:

This section of the form is titled "Photos & Documents" and contains two main areas: "Property Images" and "Legal Verification Documents". Each area has a dashed border and contains the text "Drag & drop files here, or click to browse" followed by supported file types: "image/jpeg,image/png,image/png" for images and "application/pdf,image/jpeg,image/png" for documents. At the bottom of this section is a blue button labeled "List Property".

4.Tenant Dashboard:

Tenant can check the details of the properties:



Tenant can enquire about the property through texting to owner :

The screenshot shows the 'Send an enquiry' form. At the top, it says 'Send an enquiry' and 'This will create a new enquiry entry and show up as a notification on the owner dashboard.' A 'Tenant: vinay' label is in the top right. Below the header, there is a 'Message' section with a large text input area containing the placeholder text 'Is it furnished?'. Below the input area, there are three toggle switches: 'Is it furnished?', 'What is the deposit amount?', and 'Is food included?'. At the bottom left is a blue 'Enquire' button, and at the bottom right is a 'Back to search' link.

5.Communication :

Owner can respond to the enquiry by tenant :

Enquiries

Tenants who enquire will show up here.

2 new

vinay

rev hotels

Is parties allowed?

Call

Email

Mark read

24/03/2026, 09:31:03

New

vinay

Moon light hostels

Is it furnished?

Call

Email

Mark read

24/03/2026, 09:07:26

New

CHAPTER – 5

CONCLUSION AND SCOPE

CONCLUSION:

The Home Rental & PG Finder represents a significant advancement in the digital real estate and shared accommodation market by directly addressing the systemic "trust deficit" that plagues traditional listing platforms. By shifting the paradigm from a passive directory to an active, system-verified marketplace, this project provides a secure and efficient ecosystem for both tenants and property owners.

The successful implementation of the MERN stack (MongoDB, Express, React, and Node.js) has resulted in a high-performance, mobile-first application capable of handling complex functional requirements, from secure JWT-based authentication to real-time communication. The integration of a mandatory Admin-led verification process serves as a critical gatekeeper, ensuring that every listing is backed by legitimate documentation and reducing the prevalence of fraudulent advertisements.

Furthermore, the project's "intelligent" core—driven by an AI-powered matching algorithm—transforms the user experience by providing personalized, behavioral-based recommendations rather than static search results. Features such as the automated "Freshness Filter" and responsiveness-based owner ranking ensure that the platform remains self-regulating and consistently reliable.

In summary, the Home Rental & PG Finder achieves its primary objective of making the property search process simpler, safer, and more human-centered. It stands as a robust blueprint for future rental platforms, with a scalable architecture ready to incorporate upcoming enhancements like digital rental agreements and virtual 360-degree tours. This project proves that through the strategic application of modern web technologies and rigorous validation logic, the challenges of urban housing discovery can be effectively mitigated.

FUTURE SCOPE

The future scope of the **Home Rental & PG Finder** focuses on transitioning the platform from a discovery-oriented marketplace into a fully integrated, end-to-end real estate management ecosystem. By leveraging emerging technologies, the platform aims to further automate the rental lifecycle and enhance user immersion.

1. Integrated Financial and Legal Services

Digital Rental Agreements: The platform will incorporate an automated generator for legal rental contracts, allowing tenants and owners to sign agreements digitally, thereby reducing paperwork and legal overhead.

In-App Payment Gateway: To complete the transaction loop, a secure payment

gateway will be integrated to handle token advances, security deposits, and monthly rent payments directly through the application.

2. Advanced Immersive Technologies

- **Virtual 360-Degree Tours:** To save time for both parties, the system will support virtual property inspections, allowing prospective tenants to explore a home or PG in a 360-degree environment before scheduling a physical visit.
- **Augmented Reality (AR) Furniture Placement:** Future iterations may include AR tools to help tenants visualize how their own furniture or belongings would fit within a specific room or flat.

3. AI and Data-Driven Enhancements

- **Predictive Market Analytics:** The AI matchmaker will be expanded to provide owners with pricing suggestions based on real-time market trends and demand in specific geographic areas.
- **Hyper-Personalized Neighborhood Insights:** Beyond property details, the system will integrate third-party APIs to provide tenants with detailed "livability scores," including commute times, local crime rates, and proximity to essential services like hospitals and grocery stores.

4. Community and Reputation Building

- **Tenant Reliability Scoring:** Similar to the current owner ranking system, a tenant reputation score could be implemented based on payment history and previous landlord feedback to foster mutual trust.
- **IoT Smart Home Integration:** For premium listings, the platform could integrate with IoT devices to allow owners to manage smart locks or monitor utility usage, providing a "Smart Living" experience for tenants.

REFERENCES:

Technical Stack & Tool References

- **React.js:** Documentation for building responsive, component-based user interfaces.
- **Node.js & Express.js:** Framework documentation for building scalable server-side applications and RESTful APIs.
- **MongoDB & MongoDB Atlas:** Documentation for NoSQL database management and cloud-based data storage.
- **JSON Web Tokens (JWT):** Standard for secure session management and authentication.

Security Standards: Bcrypt hashing for password encryption and input sanitization techniques for NoSQL injection prevention.

External API & Service References

- **Cloudinary / AWS S3:** Technical guides for high-resolution image storage and management.

Google Maps API / Mapbox: Documentation for integrating geolocation and interactive map services.

AI Implementation: Research on behavioral learning and recommendation algorithms used for the "AI Matchmaker" logic

sAPPENDIX :

Appendix A: User Manual

A.1 Tenant Operations

- **Registration & Login:** Users can create an account by providing a valid email and password, which triggers a secure JWT session.

Property Search: Tenants use the global search bar to query by city or landmark and apply filters for budget and property type.

Comparison Tool: Users can select up to three properties to view their amenities (e.g., Wi-Fi, Food, Laundry) in a side-by-side layout.

Inquiry Submission: Tenants send "Interest Requests" directly to owners via a secure form on the property details page.

A.2 Property Owner Operations

- **Listing Creation:** Owners must fill in mandatory fields, including location, rent, and type, and upload property images.

Verification Upload: Owners are required to upload government-issued documents to receive a "Trust Badge".

Lead Management: Owners track performance through an engagement tracker and mark properties as "Booked" to stop new inquiries.

A.3 Admin Operations

Validation Dashboard: Admins review uploaded documents against listing details using a split-screen interface.

Approval/Rejection: Admins can "Approve" a listing to make it live or "Reject" it, which notifies the owner of the necessary corrections.

Test Case ID	Feature Under Test	Input/Action	Expected Result
TC-01	User Authentication	Valid Email and Password	System issues a JWT and redirects to dashboard.
TC-02	Property Search	City: "Bangalore"	Only properties in Bangalore are displayed.
TC-03	Verification Logic	Admin clicks "Approve"	Property becomes visible to tenants with a "Verified" badge.
TC-04	Freshness Filter	31 days of inactivity	Listing is automatically hidden from search results.
TC-05	Image Upload	6MB JPEG File	System rejects upload due to 5MB size limit.

Appendix C: Non-Functional Requirements (NFR)

Performance: Search results are optimized to load within 2 seconds under 4G network conditions.

Scalability: The backend supports 1,000+ concurrent users without performance degradation.

Security: Passwords are encrypted using the bcrypt algorithm, and sensitive documents are watermarked automatically.

Reliability: The system targets 99.9% uptime with automated daily database backups.

Appendix D: Bibliography & Data Sources

•

Technical Stack: React.js, Node.js, Express.js, and MongoDB Atlas documentation.

Infrastructure: Cloudinary/AWS S3 for image hosting and Google Maps/Mapbox for location services.